Servlets: Complete Masterclass

Note : For command on running tomcat server on windows/mac refer to the last.

1. Why Do We Need Servlets?

Before understanding Servlets, first understand what happens when a browser sends a request.

When we open:

```
http://localhost:8080/hello
```

the browser is not directly saying:

```
Please call HelloServlet.java
```

A browser does not understand Java classes, Java methods, or Java objects.

It only understands **HTTP**.

So the browser sends an HTTP request like this:

```
GET /hello HTTP/1.1
Host: localhost:8080
```

This is just text written according to HTTP rules.

The important question is:

> Who receives this HTTP request?

A Java class does not automatically receive browser requests.

There must be a server process running on a port that can receive the request.

---

2. Java Can Listen on a Port

Java can do networking through the `java.net` package.

Using classes like `ServerSocket`, Java can listen on a port:

```
ServerSocket serverSocket = new ServerSocket(8080);
```

This means Java can open port `8080` and wait for incoming connections.

So technically, we can write a Java program that receives browser requests.

But the browser sends HTTP text, and if we use raw Java networking, we must handle everything manually.

For example, we would need to manually:

```
1. Read raw HTTP text
2. Parse the URL
3. Parse headers
4. Parse query parameters
5. Parse request body
6. Identify GET, POST, PUT, DELETE
7. Create response text
8. Add status code
9. Add response headers
10. Send response back to browser
11. Manage multiple users at the same time
12. Manage threads
13. Keep the server running continuously
```

Java can do networking, but building a full web application directly with low-level sockets is difficult, repetitive, and error-prone.

This is the first major problem Servlets help solve.

---

3. Normal Java Program vs Web Application

In a normal Java program, the entry point is:

```
public static void main(String[] args)
```

The program starts, executes some code, prints output, and usually finishes.

A web application is different.

A web application should not start once and stop immediately.

It should keep running continuously and wait for HTTP requests.

For example:

```
User 1 sends request
User 2 sends request
User 3 sends request
```

The application should keep responding to all these requests.

So a web application needs a **long-running server environment**.

A normal Java class alone is not enough for that.

---

4. Static Website vs Dynamic WebsiteStatic Website

Suppose we have a simple HTML file:

```
<h1>Welcome to my website</h1>
```

If the browser asks for:

```
GET /index.html
```

the server simply returns that file.

No Java logic is required.

The same request gives the same pre-written response.

This is called a **static website**.

Simple meaning:

> A static website returns already existing files.

Examples of static files:

```
HTML
CSS
JavaScript
Images
Videos
```

Dynamic Website

Now imagine this request:

```
GET /user?id=101
```

Here, the response depends on the user id.

For `id=101`, the response may be:

```
Name: Rahul
Email: rahul@test.com
```

For `id=102`, the response may be:

```
Name: Priya
Email: priya@test.com
```

Now the server needs logic.

It may need to:

```
1. Read the user id
2. Connect to a database
3. Fetch user details
4. Prepare the response
5. Send the response back
```

This is where Java becomes useful.

Java is good for writing business logic.

But there is still one important question:

> How will the HTTP request enter Java code?

This is the gap Servlets solve.

---

5. The Main Problem Before Servlets

Before Servlets, if we wanted to build a Java web application, every developer would have to write a lot of low-level networking and HTTP handling code manually.

The flow would look like this:

```
Browser
   ↓
Raw HTTP request
   ↓
Java socket code
   ↓
Manual HTTP parsing
   ↓
Manual routing
   ↓
Manual response creation
```

This is not practical for every web application.

Developers should focus on business logic, not on manually parsing HTTP again and again.

So Java needed a standard way where:

```
Low-level HTTP work is handled by a server/container
Business logic is written by the developer
```

That standard idea is called a **Servlet**.

---

6. What Servlet Did Conceptually

Servlet introduced a simple idea:

> Instead of every developer manually handling raw HTTP, let a container handle HTTP and call our Java class.

So the flow became:

```
Browser
   ↓
HTTP Request
   ↓
Tomcat
   ↓
HttpServletRequest object
HttpServletResponse object
   ↓
Our Servlet class
```

Tomcat handles the low-level HTTP work.

Our Servlet handles the application logic.

So instead of reading raw HTTP text manually, we get ready-made Java objects:

```
HttpServletRequest request
HttpServletResponse response
```

The request object gives us details like:

```
URL
HTTP method
Headers
Query parameters
Request body
Cookies
```

The response object helps us send data back to the browser.

---

7. Simplest Meaning of Servlet

A Servlet is:

> A Java class that Tomcat can call when an HTTP request comes.

In practical words:

> A Servlet is a Java class used to handle web requests and generate web responses.

Example:

```
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response) {
        // Logic for GET request
    }
}
```

The browser does not directly call `doGet()`.

Tomcat calls `doGet()` when a matching GET request comes.

---

8. Servlet Does Not Directly Listen on Port 8080

A common beginner mistake is thinking:

> Servlet is listening on port 8080.

That is not technically correct.

The correct mental model is:

```
Tomcat listens on the port.
Servlet lives inside Tomcat.
When a matching request comes, Tomcat calls the Servlet.
```

Flow:

```
Port 8080
   ↓
Tomcat is listening
   ↓
Request comes: /hello
   ↓
Tomcat checks URL mapping
   ↓
Tomcat calls HelloServlet
```

So the Servlet is not directly connected to the network.

Tomcat is the long-running server process.

Servlet is a Java object managed by Tomcat.

---

9. Browser Is the Client

When we open:

```
http://localhost:8080/hello
```

the browser acts as the client.

Its job is simple:

```
Send request → receive response
```

The browser does not care whether the backend is written in:

```
Java
Python
Node.js
PHP
Go
```

The browser only cares about the HTTP response.

From the browser's perspective:

```
I sent an HTTP request.
I received an HTTP response.
```

---

10. Server Is the Receiver

The request reaches a server process that is already running.

In the Servlet world, that server process is usually:

```
Tomcat
```

Tomcat usually runs on port:

```
8080
```

So this URL:

```
http://localhost:8080/hello
```

means:

Part	Meaning
`localhost`	My own machine
`8080`	Port where Tomcat is listening
`/hello`	Path requested by the browser

The request first reaches Tomcat.

Then Tomcat decides which Java code should handle the request.

---

11. What Is Tomcat?

Tomcat is a Java-based server that can run Java web applications.

More specifically:

> Tomcat is a Servlet Container.

This means Tomcat provides the environment where Servlets can live, run, and handle HTTP requests.

---

12. What Is a Servlet Container?

The word **container** is very important.

A container means:

> Something that creates, manages, and controls objects.

In Spring Core, we say:

```
Spring Container manages beans.
```

Similarly, in the Servlet world:

```
Servlet Container manages Servlets.
```

Tomcat is called a Servlet Container because it manages Servlet objects.

Tomcat controls:

```
1. Servlet object creation
2. init() method call
3. service() method call
4. doGet() / doPost() / doPut() / doDelete() method call
5. destroy() method call
6. Request object creation
7. Response object creation
8. Thread management
9. URL mapping
10. Sending response back to browser
```

So we do not manually create Servlet objects.

Tomcat creates and manages them.

13. Control Is Inverted

This connects with the idea of Inversion of Control.

In normal Java:

```
We create object.
We call method.
```

Example:

```
HelloServlet servlet = new HelloServlet();
servlet.doGet(request, response);
```

But in Servlet-based applications:

```
Tomcat creates the object.
Tomcat calls the method.
```

So the control is not with our code.

The control is with the container.

This is similar to the idea of **Inversion of Control**.

World	Object Managed By
Servlet world	Tomcat / Servlet Container
Spring world	Spring Container

14. What Happens When Tomcat Starts?

When Tomcat starts, it becomes a long-running server process.

It does not start, run one method, and stop.

It keeps waiting for HTTP requests.

Basic flow:

```
Tomcat starts
   ↓
Tomcat opens port 8080
   ↓
Tomcat waits for HTTP requests
   ↓
Browser sends request
   ↓
Tomcat receives request
   ↓
Tomcat finds matching application
   ↓
Tomcat finds matching Servlet
   ↓
Tomcat calls Servlet method
   ↓
Servlet prepares response
   ↓
Tomcat sends response back to browser
```

---

15. What Tomcat Does When a Request Comes

Suppose a request comes like this:

```
GET /myapp/hello
```

Tomcat checks:

```
Which application?  → myapp
Which URL?          → /hello
Which Servlet?      → HelloServlet
Which method?       → doGet()
```

Then Tomcat calls something conceptually similar to:

```
helloServlet.doGet(request, response);
```

Tomcat acts as the bridge between:

```
HTTP world and Java world
```

The browser sends HTTP.

Tomcat converts that HTTP request into Java-friendly objects.

Then our Servlet uses those objects.

---

16. Does Tomcat Create Servlet Object on Startup?

Usually, Tomcat creates the Servlet object when the first matching request comes.

Example:

```
First request to /hello
   ↓
Tomcat creates HelloServlet object
   ↓
Tomcat calls init()
   ↓
Tomcat calls doGet()
```

After that, the same Servlet object can handle future requests.

But we can also configure Tomcat to load a Servlet during application startup.

So there are two possibilities:

Behavior	Meaning
Default behavior	Servlet loads on first request
Startup loading	Servlet loads when the application starts

External Tomcat and WAR Deployment

17. What Is External Tomcat?

In traditional Java web development, Tomcat is installed separately.

The flow is:

```
1. Install Tomcat on your machine/server
2. Build the Java web application as a WAR file
3. Put the WAR file inside Tomcat
4. Start Tomcat
5. Tomcat runs the application
```

This is called using **external Tomcat**.

Mental model:

```
Application goes inside Tomcat.
```

---

18. What Is a WAR File?

WAR means:

```
Web Application Archive
```

A WAR file is a packaged Java web application.

Just like a normal Java project can be packaged as:

```
myapp.jar
```

a traditional Java web application is packaged as:

```
myapp.war
```

A WAR file can contain:

```
Compiled Java classes
Servlet classes
HTML/CSS/JS files
```

```
web.xml
Libraries
Configuration files
```

Simple mental model:

```
WAR = complete web application package
```

External Tomcat knows how to run WAR files.

---

19. One Tomcat Can Run Multiple Web Applications

External Tomcat can host multiple Java web applications at the same time.

Example:

```
External Tomcat
 ├── app1
 │    └── HelloServlet
 │
 ├── app2
 │    └── LoginServlet
 │
 └── app3
      └── ProductServlet
```

Each application can have its own Servlets.

If a request comes like:

```
/app1/hello
```

Tomcat sends it to the Servlet inside `app1`.

If a request comes like:

```
/app2/login
```

Tomcat sends it to the Servlet inside `app2`.

This is another reason Tomcat is called a container.

It can contain and manage multiple web applications.

---

20. Starting External Tomcat

Go inside your external Tomcat folder.

Example on Mac/Linux:

```
cd ~/Downloads/apache-tomcat-10.1.x
```

Inside this folder, you should see folders like:

```
bin
conf
logs
webapps
```

Check Tomcat Port

Open this file:

```
conf/server.xml
```

Find the connector configuration:

```
<Connector port="8080" protocol="HTTP/1.1"
           connectionTimeout="20000"
           redirectPort="8443" />
```

This means Tomcat will listen on port `8080`.

If it is using another port, change it to:

```
port="8080"
```

Then save the file.

Give Permission on Mac/Linux

On Mac/Linux, give permission to run Tomcat scripts:

```
chmod +x bin/*.sh
```

This step is not required on Windows.

Start Tomcat in Foreground

Foreground mode is useful while teaching because logs are visible directly.

On Mac/Linux:

```
./bin/catalina.sh run
```

If everything is fine, you should see something like:

```
Server startup in ... milliseconds
```

Now open:

```
http://localhost:8080
```

You should see the Tomcat welcome page.

Start and Stop Tomcat in Background

On Mac/Linux:

```
./bin/startup.sh
```

To stop Tomcat:

```
./bin/shutdown.sh
```

On Windows:

```
bin\startup.bat
```

To stop Tomcat:

```
bin\shutdown.bat
```

If Port 8080 Is Already Busy

On Mac/Linux, check what is using port `8080`:

```
lsof -i :8080
```

You may see a Java process using the port.

Kill that process:

```
kill -9 <process-id>
```

Then start Tomcat again.

---

21. Deploying WAR in External Tomcat

After building the project, copy the WAR file:

```
target/servlet-crud.war
```

into Tomcat's `webapps` folder.

Mac/Linux

```
cp target/servlet-crud.war ~/tools/apache-tomcat-10/webapps/
```

Windows

Copy:

```
target\servlet-crud.war
```

to:

```
C:\tools\apache-tomcat-10\webapps
```

Now start Tomcat.

Tomcat will automatically extract:

```
servlet-crud.war
```

into:

```
webapps/servlet-crud
```

Now open:

```
http://localhost:8080/servlet-crud/
```

If the app is configured correctly, you should see:

```
Servlet CRUD App is running
```

Servlet Dependency Scope

22. Why Is Servlet Dependency Scope `provided`?

In a Servlet project, we use Servlet classes like:

```
HttpServlet
HttpServletRequest
HttpServletResponse
```

These classes are needed to compile our code.

But at runtime, these Servlet classes are already provided by Tomcat.

External Tomcat has its own `lib` folder:

```
apache-tomcat-10
 ├── bin
 ├── conf
 ├── lib
 ├── logs
 ├── webapps
```

Inside Tomcat's `lib`, Tomcat already has the Servlet-related classes needed to run web applications.

So Maven needs the Servlet dependency only during compilation.

At runtime, Tomcat provides it.

That is exactly what `provided` means.

Simple meaning:

```
I need this dependency while compiling my code,
but do not package it inside my final WAR,
```

```
because the runtime environment will provide it.
```

In this case:

```
Compile time → Maven needs Servlet API
Runtime      → Tomcat provides Servlet API
```

So the dependency should not be bundled inside:

```
servlet-crud.war
```

---

23. What Happens If We Do Not Use `provided`?

If we remove:

```
<scope>provided</scope>
```

then Maven uses the default scope:

```
compile
```

That means Maven may package the Servlet API inside the WAR file.

But Tomcat already has the Servlet API.

Now two copies may exist:

```
One copy inside Tomcat
One copy inside the application WAR
```

This can create classloading conflicts or confusing runtime issues.

In container-based applications, container APIs like Servlet API should usually be provided by the container, not packaged by the application.

Query Parameters in First Servlet Example

24. Why Use Query Parameters Instead of POST Body?

In the first Servlet example, query parameters are easier to use.

For example:

```
/users?name=Rahul&email=rahul@test.com
```

Using a JSON request body in Servlet requires extra work.

We would need to understand:

```
1. How to read the request body manually
2. How to parse JSON manually
3. How to add Jackson or Gson dependency
4. How to map JSON to a Java object
5. How to handle invalid JSON
```

So for the first Servlet example, query parameters keep the focus on the main concept:

```
How Tomcat receives a request and calls our Servlet.
```

Later, JSON request body handling can be introduced separately.

Servlet Lifecycle

25. What Is Servlet Lifecycle?

Servlet lifecycle means the complete journey of a Servlet object.

The basic lifecycle is:

```
Servlet object creation
   ↓
init()
```

```
   ↓
service()
   ↓
doGet() / doPost() / doPut() / doDelete()
   ↓
destroy()
```

Tomcat controls this lifecycle.

We do not manually create or destroy Servlet objects.

---

26. `init()` Method

`init()` is called when the Servlet object is initialized.

Usually:

```
First request comes
   ↓
Tomcat creates Servlet object
   ↓
Tomcat calls init()
```

By default, Tomcat usually creates the Servlet object on the first matching request.

Example:

```
First GET /lifecycle

Constructor runs
   ↓
init() runs
   ↓
service() runs
   ↓
doGet() runs
```

After that, if another request comes:

```
Second GET /lifecycle
   ↓
```

```
service() runs
   ↓
doGet() runs
```

The constructor and `init()` do not run again for every request.

---

27. Use of `init()`

`init()` is used for one-time setup work.

Examples:

```
Load configuration
Create expensive resources
Initialize helper objects
Open connection pool
Prepare cache
```

---

28. `service()` Method

`service()` is called for every request.

For example:

```
GET /lifecycle      → service()
POST /lifecycle     → service()
PUT /lifecycle      → service()
DELETE /lifecycle   → service()
```

But when we extend `HttpServlet`, we usually do not write all logic inside `service()`.

Why?

Because `HttpServlet` already has a `service()` method that checks the HTTP method and forwards the request to the correct method.

Conceptually:

```
if (requestMethod is GET) {
    call doGet();
}

if (requestMethod is POST) {
    call doPost();
}

if (requestMethod is PUT) {
    call doPut();
}

if (requestMethod is DELETE) {
    call doDelete();
}
```

So the internal flow is:

```
service()
   ↓
Checks HTTP method
   ↓
Calls doGet() / doPost() / doPut() / doDelete()
```

In real Servlet code, we usually override:

```
doGet()
doPost()
doPut()
doDelete()
```

We usually do not override `service()` unless we have a specific reason.

29. `doGet()`, `doPost()`, `doPut()`, and `doDelete()`

`doGet()`

`doGet()` handles HTTP GET requests.

Usually used for reading/fetching data.

`doPost()`

`doPost()` handles HTTP POST requests.

Usually used for creating new data.

`doPut()`

`doPut()` handles HTTP PUT requests.

Usually used for updating existing data.

`doDelete()`

`doDelete()` handles HTTP DELETE requests.

Usually used for deleting data.

---

30. `destroy()` Method

`destroy()` is called when the Servlet is being removed from service.

This can happen when:

```
Tomcat stops
Application is undeployed
Application is reloaded
New WAR replaces old WAR
```

Use of `destroy()`:

```
Close resources
Release memory-heavy objects
Stop background tasks
Flush pending data
```

Important point:

```
destroy() is called once before the Servlet is removed.
```

After `destroy()`, the Servlet object becomes eligible for garbage collection.

---

31. Most Important Lifecycle Point

Usually, Tomcat creates one Servlet object and reuses it for multiple requests.

So the flow is not:

```
Every request → new Servlet object
```

The better mental model is:

```
One Servlet object → many requests
```

That is why we should be careful with instance variables inside Servlets, especially when multiple users are sending requests at the same time.

---

32. How to See `destroy()` Output

To see `destroy()` output, stop Tomcat.

On Mac/Linux:

```
./bin/shutdown.sh
```

On Windows:

```
shutdown.bat
```

When Tomcat stops, it removes the Servlet from service and calls `destroy()`.

Embedded Tomcat in Spring Boot

33. What Is Embedded Tomcat?

In traditional Servlet applications:

```
Application goes inside Tomcat.
```

But in Spring Boot, we usually do not install Tomcat separately.

Spring Boot brings Tomcat inside the application through:

```
spring-boot-starter-web
```

When we run:

```
public static void main(String[] args) {
    SpringApplication.run(MyApp.class, args);
}
```

Spring Boot starts the application and also starts embedded Tomcat internally.

So in Spring Boot:

```
Tomcat is inside the application.
```

34. External Tomcat vs Embedded Tomcat

Traditional Java Web App

```
Install Tomcat
   ↓
Build WAR
   ↓
Deploy WAR inside Tomcat
   ↓
Start Tomcat
   ↓
Application runs
```

Mental model:

```
Application is deployed into the server.
```

Spring Boot Web App

```
Run main()
   ↓
Spring Boot starts
   ↓
Embedded Tomcat starts
   ↓
Application is ready
```

Mental model:

```
Server starts inside the application.
```

---

35. Why Spring Boot Uses Embedded Tomcat

Spring Boot makes Java web applications easier to run.

Without embedded Tomcat:

```
Install Tomcat
Configure Tomcat
Build WAR
Deploy WAR
Start server
Debug deployment issues
```

With embedded Tomcat:

```
Run main()
Application starts
Server starts
API is ready
```

Spring Boot did not remove the Servlet Container concept.

It simply made it easier to use.

Internally, Tomcat is still there.

From Servlets to Spring MVC

36. Traditional Servlet Style

In a traditional Servlet application, we may create different Servlets for different features.

Example:

```
/users       → UserServlet
/products    → ProductServlet
/orders      → OrderServlet
```

Flow:

```
Browser/Postman
   ↓
Tomcat
   ↓
Specific Servlet
   ↓
Business logic
```

Example:

```java
@WebServlet("/users")
public class UserServlet extends HttpServlet {
}
```

Here, Tomcat checks the URL and directly calls the matching Servlet.

This works, but as the application grows, we may end up creating many Servlets.

Each Servlet may handle:

```
Request reading
Response writing
Validation
Routing
Error handling
Business flow
```

Slowly, the code becomes difficult to manage.

---

37. The Spring MVC Idea

Spring MVC also uses the Servlet concept.

But instead of asking us to manually create many Servlets, Spring MVC gives us one central Servlet.

That central Servlet is called:

```
DispatcherServlet
```

This is the heart of Spring MVC.

The idea is simple:

> Let all incoming requests first come to one central Servlet, and then that Servlet will decide which controller method should handle the request.

So in Spring MVC, the flow becomes:

```
Browser/Postman
   ↓
Tomcat
   ↓
DispatcherServlet
   ↓
Controller
   ↓
Service
   ↓
Response
```

---

38. Traditional Servlet vs Spring MVC

Traditional Servlet Style

```
/users       → UserServlet
/products    → ProductServlet
/orders      → OrderServlet
```

Here, we manually create multiple Servlets.

Spring MVC Style

```
/users       ┐
/products    ├── DispatcherServlet ──→ Correct Controller Method
/orders      ┘
```

Here, requests first come to `DispatcherServlet`.

Then `DispatcherServlet` finds the correct controller method.

Example:

```
@RestController
public class UserController {

    @GetMapping("/users")
    public String getUsers() {
        return "All users";
    }
}
```

The browser is still sending an HTTP request.

Tomcat is still receiving that request.

But instead of directly calling our custom `UserServlet`, Tomcat calls Spring's `DispatcherServlet`.

Then `DispatcherServlet` calls the correct controller method.

---

39. Important Mental Model for Spring MVC

A Spring MVC controller is not directly called by the browser.

The browser does not know about:

```
UserController
getUsers()
@GetMapping
```

The browser only sends:

```
GET /users
```

The actual flow is:

```
Browser sends HTTP request
   ↓
Tomcat receives request
   ↓
Tomcat calls DispatcherServlet
   ↓
DispatcherServlet checks Spring MVC mappings
   ↓
Correct controller method is called
   ↓
Response goes back
```

---

40. Why Spring MVC Is Powerful

With traditional Servlets, we manually handle many things.

For example:

```
Read request parameters
Call service class
Prepare response
Convert object to JSON
Handle errors
Manage dependencies
```

Spring MVC simplifies this by giving us:

```
@Controller / @RestController
@GetMapping / @PostMapping
@RequestParam
@RequestBody
@ResponseBody
Automatic JSON conversion
Cleaner Controller-Service-Repository separation
```

But internally, the foundation is still Servlet.

Spring MVC did not remove Servlet.

Spring MVC built a better framework on top of Servlet.

Final Mental Model

The complete Servlet request flow is:

```
Browser sends HTTP request
   ↓
Tomcat receives request
   ↓
Tomcat identifies application
   ↓
Tomcat identifies Servlet
   ↓
Tomcat creates request and response objects
   ↓
Tomcat calls Servlet method
   ↓
Servlet writes response
   ↓
Tomcat sends HTTP response back to browser
```

Remember this:

```
Browser speaks HTTP.
Java writes business logic.
Tomcat connects the HTTP world with the Java world.
Servlet is the Java class Tomcat calls.
```

Do not think your Java class is directly listening to the browser.

The correct understanding is:

```
Tomcat listens.
Tomcat receives the HTTP request.
Tomcat converts it into Java-friendly objects.
Tomcat calls your Servlet.
Your Servlet writes the response.
Tomcat sends it back to the browser.
```

This is the foundation of Java web development.

Once this is clear, Spring MVC and Spring Boot controllers become much easier to understand.

Running Tomcat 10macOS / Linux

Go inside Tomcat folder:

```
cd ~/Downloads/apache-tomcat-10.1.x
```

Give permission to scripts:

```
chmod +x bin/*.sh
```

Run Tomcat in foreground:

```
./bin/catalina.sh run
```

Open in browser:

```
http://localhost:8080
```

Stop Tomcat:

```
Control + C
```

Run Tomcat in background:

```
./bin/startup.sh
```

Stop background Tomcat:

```
./bin/shutdown.sh
```

Windows

Go inside Tomcat folder:

```
cd C:\apache-tomcat-10.1.x
```

Run Tomcat in foreground:

```
bin\catalina.bat run
```

Open in browser:

```
http://localhost:8080
```

Stop Tomcat:

```
Control + C
```

Run Tomcat in background:

```
bin\startup.bat
```

Stop background Tomcat:

```
bin\shutdown.bat
```

If Port 8080 Is Busy macOS / Linux

```
lsof -i :8080
kill -9 PROCESS_ID
```

Windows

```
netstat -ano | findstr :8080
taskkill /PID PROCESS_ID /F
```